∞ AZURE DEVOPS

Code Generation Pipelines

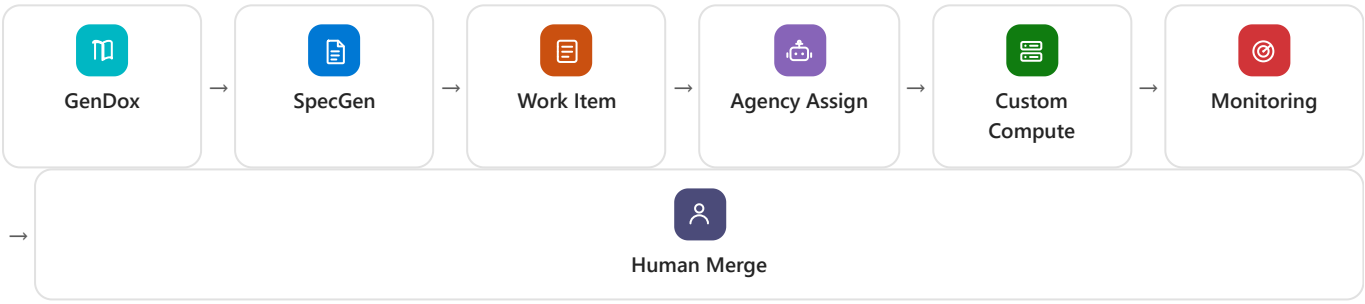
How We Built It — And How You Can Set It Up Too

GenDox → SpecGen → Work Item → Agency Assign → Custom Compute → Monitoring → Human Merge

Organization	dev.azure.com/microsoft
Project	WindowsProtocolTestSuites
Pipeline folder	`\WindowsProtocolTestSuites\Code Generation`
Format	Step-by-step runbook, in data-flow order
Prepared for	James Omitiran (jomitiran@microsoft.com)

Contents

- Step 1 — Prerequisites: CLI access
- Phase 1 — GenDox: the source of truth (Steps 2–4)
- Phase 2 — Specification Generation (Steps 5–10)
- Phase 3 — Work Item Creation (Steps 11–14)
- Phase 4 — Agency Assignment (Steps 15–19)
- Phase 5 — Agency Custom Compute (Steps 20–24)
- Phase 6 — Monitoring & Validation (Steps 25–32)
- Phase 7 — Human review & merge (Step 33)
- Appendix — Shared infra, pipeline table, CLI reference



Step 1 — Prerequisites: get CLI access

```
az extension add --name azure-devops        # az 2.77.0, azure-devops 1.0.2
az login                                     # sign in with your @microsoft.com account
az devops configure --defaults organization=https://dev.azure.com/microsoft \
                               project=WindowsProtocolTestSuites
```

- `az account show` must return tenant `72f988bf-86f1-41af-91ab-2d7cd011db47` (Microsoft).
- `az rest` calls need `--resource 499b84ac-1321-427f-aa17-267ca6975798` (the Azure DevOps AAD resource ID).
- On Windows, `az rest` can crash on non-cp1252 characters — pipe to `--output-file <path>` and set `PYTHONIOENCODING=utf-8` first.
- Requires Contributor on `WindowsProtocolTestSuites`, plus read access to `SpecificationGenerationv2` and `Code Generation WPTS`.

Phase 1 — GenDox: the source of truth

Steps 2–4

Why this phase exists: the authoritative copy of each protocol spec lives in GenDox — hosted in a **different** Azure DevOps org (`ointprotocol` / `GenDoxDataProduction`) than the pipelines (`microsoft`). Every protocol has a Book at a path like `Teams/Windows/Published/Books/MS-SMB2`. This phase sets up the cross-org auth needed before any diffing runs.

2 **Provision a managed identity for cross-org reads.**
Create user-assigned managed identity `protocopilotmanagedidentitymvp`, grant it **Contributor** on `GenDoxDataProduction` (org `ointprotocol`).

3 **Create the Azure RM service connection.**
In `WindowsProtocolTestSuites`, create **SpecGen-GenDox-MI** using workload identity federation (WIF), bound to that managed identity (id `1f7e45a8-43fa-41de-afa9-65339fc63952`).

4 **Mint a token at run time — never a stored secret.**
An `AzureCLI@2` task authenticates via this connection and runs `az account get-access-token --resource 499b84ac-…` to get `GENDOX_GIT_TOKEN` fresh on every run.

Phase 2 — Specification Generation

Pipeline 188962 · Steps 5–10

> **Why:** protocols change over time. SpecGen watches for those changes and turns them into two outputs — a partner-facing comparison report, and an internal diff that later becomes work items.

5. **Create the repo and pipeline.**
   Repo `SpecificationGenerationv2` (branch main), YAML `azure-pipelines.yml` + `ci/specgen-steps.yml`. Pipeline id **188962**, pool `windows-latest`.

6. **Define the protocol registry.**
   `supportedProtocols`: MS-SMB2, MS-FSA, MS-RDPBCGR. Onboarding = add the code here (must exist as a GenDox Book + be published on Learn.microsoft.com). `protocolCode` gates which run, at compile time via `${{ each }}/${{ if }}`.

7. **Schedule it.**
   Cron `0 16 * * 1-5` — 8am PST weekdays, `always: true` (state tracking, Step 9, decides if there's real work).

8. **Run the partner-facing (LMC) flow.**
   `--source lmc --max-previous 15`, sourced from public Learn.microsoft.com. Publishes comparison artifacts. Never touches GenDox content, never creates work items.

9. **Run the internal (GenDox) flow.**
   Hydrate prior state, then `--source gendox` via the Phase-1 connection. New commits found → writes `gendox-run.json`, sets `gendoxHasChanges=true` — the flag that drives Phase 3.

10. **Validate one protocol before trusting the schedule.**
    ```
    az pipelines run --id 188962 --parameters protocolCode=MS-SMB2
    ```

Phase 3 — Work Item Creation

tools/Create-AdoWorkItems.ps1 · Steps 11–14

Why: a raw diff isn't actionable — it needs to become a tracked, assignable unit of work with enough context that a human or an AI agent can act on it without re-reading the whole spec.

11 **Only runs when there's something new.**

Gated on `gendoxHasChanges == true`. Reads `manifest.json` for the latest-vs-previous pair, then the per-section `flagged-sections-*.json`.

12 **Create one ADO Task per flagged section — in project `OS`, not WindowsProtocolTestSuites.**

Field	Value
Title	`[SpecGen] <Protocol> v<ver>: Section <X.X.X> - <Title> (<ChangeType>)`
Description	HTML: review header + unified diff (or full content) + footer requiring an `ADDED TESTS:` line in the eventual PR. Capped under ADO's 8000-char limit.
Tags	`SpecGen; WPTS-AI-TASK; WPTS AI Task; <Protocol>; v<ver>; <ChangeType>[; commit:<hash>]`
Area Path	`OS\ImPaCT\TEC\Data and Protocols`
Assigned To	a **human** reviewer by default — not the AI agent yet (that's Phase 4)

Dedup: skips if a work item with the exact title already exists.

13 **Persist state so the next run knows where it left off.**

`commit_tracker` writes `state/gendox/<protocol>.json`, then `PUT`s it into **CodeGenerationStateGroup** via REST — the build identity needs **Administrator** on that group, or this 403s.

14 **Publish internal-only audit artifacts.**

`comparison-artifacts.zip` from the GenDox flow, labelled INTERNAL — never shared externally (unlike the LMC artifacts from Step 8).

Phase 4 — Agency Assignment

Pipeline 189142 · Steps 15–19

Why: work items from Phase 3 just sit there until something notices them, prepares a branch, and formally hands them to the AI agent (Agency). That's this pipeline's entire job.

15 **Create the repo and pipeline.**
Repo `Code Generation WPTS`, YAML `.azuredevops/pipelines/wpts-ai-codegen.yml`. Pipeline id **189142**, `trigger: none` (schedule only: `0 16 * * *`, daily 8am PST). Pool `TestSuiteBuildESPoolTME-CentralUS`.

16 **Wire up shared config.**
Service connection **AgencyWPTS**; variable group **CodeGenerationGroup** (PAT secret, ServicePrincipal, PipelineForTestID=190320, UserGroup, AllowedSpecGenProtocols).

17 **Query for pending AI work items.**
`query-work-items.ps1` runs one WIQL query per tag variant (`WPTS AI Task`, `WPTS-AI-TASK`, `WPTS AI TASK`) under the area path, State in (Proposed, Committed). Writes results to a temp file (ADO variables cap at 32KB).

18 **Process each pending item.**
`process-work-items.ps1`: (1) create branch `ai-work/{id}` from main, (2) set state → **Started**, (3) tag + assign to the AI identity — using the disambiguated string `Agency <e329845e- … @72f988bf- … >`, not the bare name "Agency" (a second, unrelated identity literally named "agency" makes plain-name assignment ambiguous and ADO rejects it). Failure → reset to Proposed + explanatory comment.

19 **This assignment is the actual trigger.**
Assigning the work item to Agency's identity is what makes Agency pick up the task (Way A — work-item assignment, per the policy file in Phase 5). Nothing else invokes Agency directly.

Phase 5 — Agency Custom Compute

Pipeline 195772 · Steps 20–24

Why: Agency's default managed compute was failing to clone this repo. We stood up a customer-owned compute pipeline that clones the repo itself and hands control back to Agency's own task — Agency still drives the coding agent; we just own the environment underneath it.

20 **Author the pipeline in the repo Agency operates on.**

`.azuredevops/pipelines/agency-custom-compute.yml` on branch `ai-work/agency-custom-compute` — a hard location constraint. Extends `v1/1ES.Agency.PipelineTemplate.yml` @ `refs/tags/canary` (Public Preview — expect churn). `useAgencyManagedPool: true`, `os: windows`.

21 **Self-clone instead of relying on built-in sync.**

`skipSourceSync: true` + a preAgentSteps block: `git -c http.extraHeader="AUTHORIZATION: bearer $(System.AccessToken)" clone $repo $dst` (never embed a token in the URL).

22 **Create the pipeline; set the 3 queue-time-only variables.**

Pipeline id **195772**. Must be set only in the pipeline UI, never YAML:

Variable	Value
`Agency.Consent.WindowsProtocolTestSuites`	`true`
`Agency_Context`	*empty — populated by Agency*
`system.connection.accessTokenScope`	`vso.agentpools vso.build_execute vso.code vso.packaging`

23 **Tell Agency to use this pipeline.**

`.azuredevops/policies/agency-preferences.yml` (main branch) with `projectId`, `pipelineId: 195772`, `pipelineTrialMode: true` — only work items tagged `agency:pipelineTrialMode=true` route here until validated.

24 **What Agency does once it has compute.**

Works the `ai-work/{id}` branch, opens a **draft** PR (`[WIP]` title, placeholder description) while coding, then **publishes** it (draft→active) when done — the signal Phase 6 waits on.

⚠ **Open item**

The custom clone step doesn't yet check out the correct `ai-work/{id}` branch on its own. The branch name is expected in the `Agency_Context` JSON, but the field name isn't documented — confirm with the Agency team or inspect a real run's context first.

Phase 6 — Monitoring & Validation

monitor-copilot-tasks.ps1 (3rd step of pipeline 189142) · Steps 25–32

> **Why:** an AI agent opening a PR isn't "done" — someone has to actually run the tests it claims to have added and hold the line on quality. This phase is the automated version of that someone.

25 **Resolve which items are still genuinely in-flight.**
Re-fetches current state; keeps only `State=Started`, assigned to Agency/GitHub Copilot, not already tagged `AgentCompleted`.

26 **Resolve the correct PR, not a stale one.**
Computes an "agent-assignment baseline" (last assignment time, from the full paged update history). PR links from before that baseline are ignored. If assigned >24h with no PR comment, checks whether pipeline 195772 is still actively running for that PR and waits for it.

27 **Wait for the PR to actually be ready.**
Polls until `isDraft` flips to false — Agency's authoritative "ready for testing" signal.

28 **Get the list of tests to run.**
Parses `ADDED TESTS: {names}` from the PR description. Missing? Posts the **literal text token** `@<Agency>` (identity-link mentions are silently ignored by the bot) asking for it, waits up to 20 min. Declares `NONE`? Escalates for manual review instead of validating nothing.

29 **Trigger targeted validation.**
Runs pipeline 190320 (`FileServer_Standard_Integration_DotNetCoreTME_Azure_Regression`) on the PR branch, scoped to the named tests, polls up to 90 min.

30 **Read real results, trust nothing else.**
Pulls actual Test Results via REST. Unreadable results or tests that never actually ran → refuses to mark complete, escalates instead.

31 **All targeted tests pass → done.**
Tags the work item **AgentCompleted**.

32 **Any test fails → remediation loop.**
One consolidated PR comment (agent mentioned once, every failure's name/error/stack), then: wait for acknowledgement → wait for fix comment → wait for the commit to actually advance → re-run all tests. Up to 10 rounds. No progress, no response, or non-convergence → escalate with a real notifying `@<team-guid>` mention.

Phase 7 — What happens at the end

Step 33

33 **Human review and merge.**

This chain does **not** auto-complete or merge pull requests. Once tagged `AgentCompleted`, the PR is validated and ready — but final code review and merge remain a manual step by the WPTS team. Everything above exists to make that review cheap: by the time a human looks at it, the diff traces to a real spec change, it's linked to a tracked work item, and its claimed tests have already been proven to pass.

Appendix — Reference

Shared infrastructure

SpecGen-GenDox-MI — Azure RM (WIF) · 1f7e45a8-43fa-41de-afa9-65339fc63952
Phase 1, consumed in Phase 2 — cross-org GenDox reads.

AgencyWPTS — Azure RM · 6ed02e51-da3d-4f62-8ccd-e38adb8d045a
Phase 4 — Agency API authentication.

CodeGenerationStateGroup — Variable group · id 6615
Phase 3, Step 13 — per-protocol GenDox commit-tracking state.

CodeGenerationGroup — Variable group · id 6386
Phase 4, Step 16 — PAT, ServicePrincipal, PipelineForTestID, UserGroup.

⚠ **Cleanup candidate**

As of 2026-08-19, `CodeGenerationGroup` still carries leftover `gendoxState_*` values from before `CodeGenerationStateGroup` was split out. The pipeline only reads/writes the latter now — worth confirming with whoever did the split (Lucien Makutano).

Pipeline definitions at a glance

ID	Name	Trigger	Pool	Repo
188962	Specification Generation	Cron, weekdays 8am PST	windows-latest	`SpecificationGenerationv2`
189142	Agency Code Generation	Cron, daily 8am PST	`TestSuiteBuildESPoolTME-CentralUS`	`Code Generation WPTS`
195772	Agency Custom Compute	None — invoked by Agency	Agency-managed pool	`WindowsProtocolTestSuites`

Quick CLI reference

```
az devops configure --defaults organization=https://dev.azure.com/microsoft
project=WindowsProtocolTestSuites
az pipelines list --folder-path "\WindowsProtocolTestSuites\Code Generation" -o table
az pipelines show --id 195772 -o json
```

```
az pipelines variable-group list --group-name CodeGenerationGroup -o json
az devops service-endpoint list -o table
az pipelines run --id 188962 --parameters protocolCode=MS-SMB2
```

Generated from live Azure DevOps pipeline and script content via `az devops` CLI, 2026-08-20. Organization: dev.azure.com/microsoft · Project: WindowsProtocolTestSuites.